Intel Xeon Phi

Sparsh Mittal

© Sparsh Mittal

Acronyms

- FMA : fused multiply-add
- QFMA : quad FMA
- SP/DP : single/double precision
- VNNI : vector neural network instructions
- GEMM: General matrix-multiply

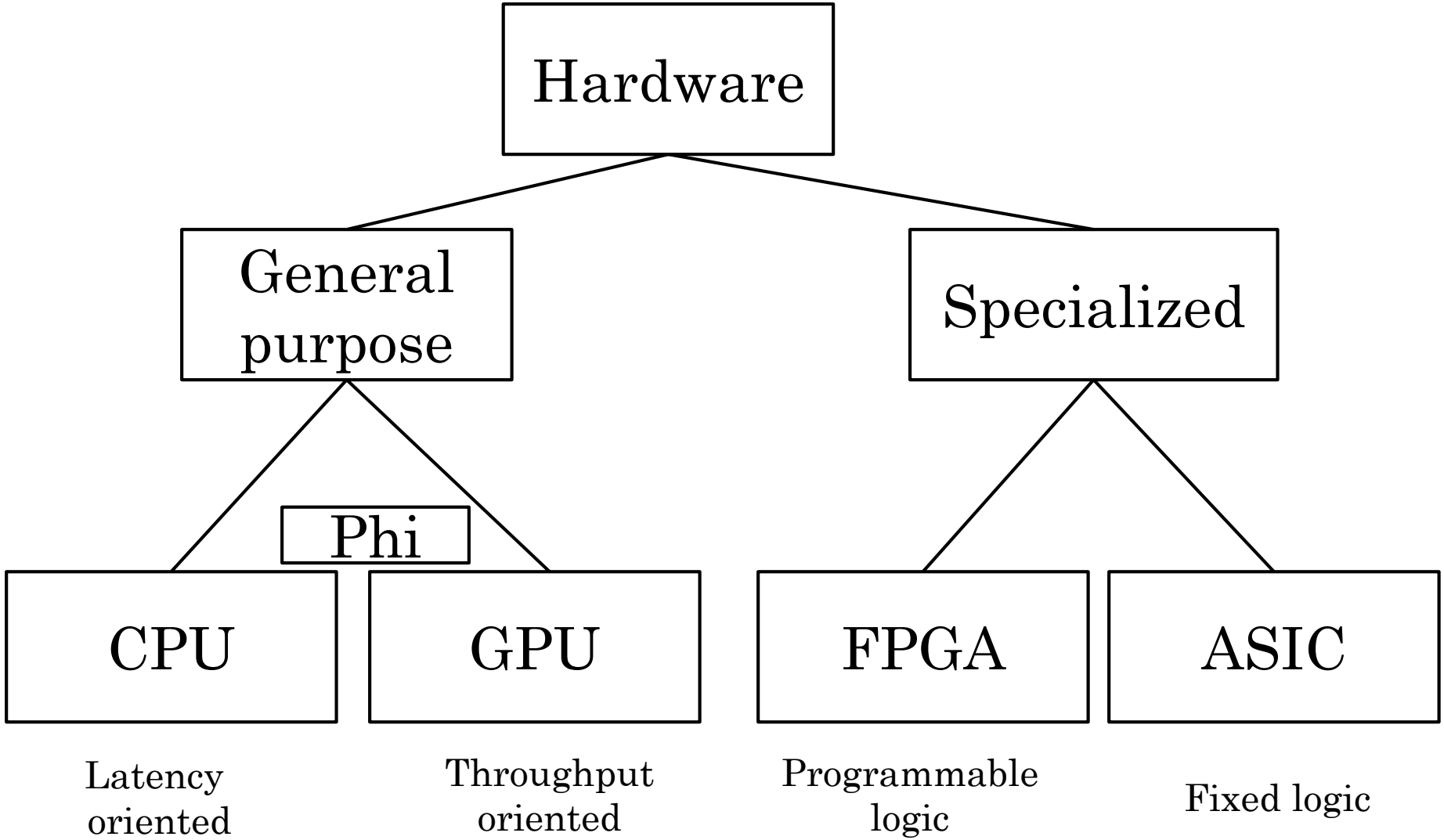
© Sparsh Mittal

Vectorization via SIMD: History

Year	Registers	Instruction Set	
~1997	80-bit	MMX	Integer SIMD (in x87 registers)
~1999	128-bit	SSE1	SP FP SIMD (xMM0-8)
~2001	128-bit	SSE2	DP FP SIMD (xMM0-8)
---	128-bit	SSEx	
~2010	256-bit	AVX	DP FP SIMD (yMM0-16)
~2012	512-bit	Phi	
~2014	512-1024-bit	(Haswell)	

© Sparsh Mittal

Landscape of computing systems



© Sparsh Mittal

Brief introduction to Xeon Phi

- Brings together parallel processing power of a many-core computing unit and programming ease of CPUs
- In June 2018 list of Top500 supercomputers, 19 supercomputers use Phi as the main processing unit and seven supercomputers use Phi as the co-processor
- Supports x86 ISA. Cores are simpler than Xeon cores:
  - Smaller cache (no shared last-level cache)
  - More for floating point operations (e.g. widened SIMD unit)

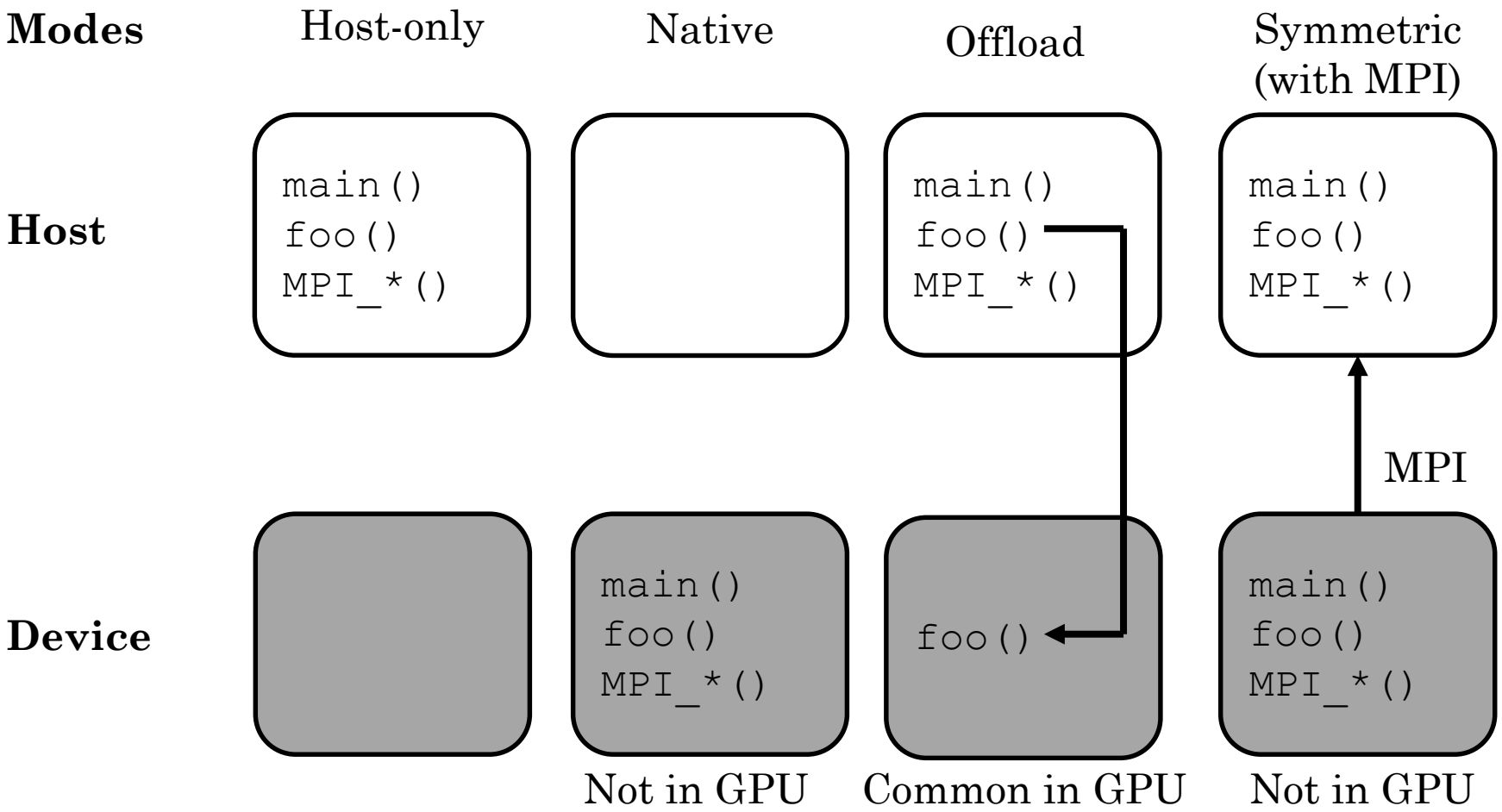
© Sparsh Mittal

Features of Phi

- Phi can run C/C++, Fortran
- Parallel programming on Phi
  - Using OpenMP, Pthreads, Intel TBB, Intel Cilk plus
- Support for MPI
- Optimization techniques on Phi is same as that on CPU

© Sparsh Mittal

Modes of operation of Phi



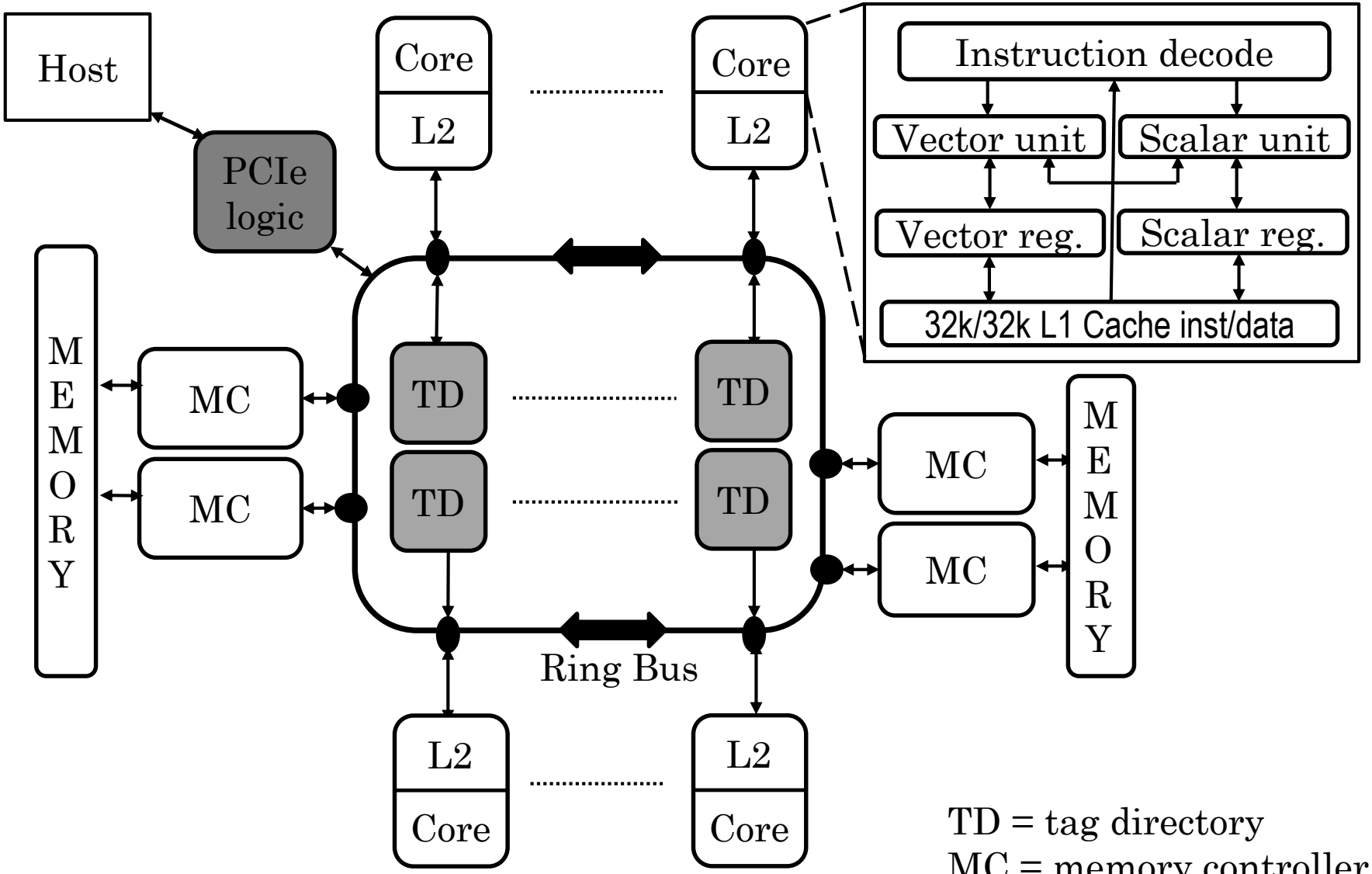
© Sparsh Mittal

Phi product generations

- Knights Ferry: only research prototype
- Knights Corner (KNC):
- Knights Landing (KNL):
- Knights Hill: cancelled
- Knights Mill (KNM): specially designed for machine learning

© Sparsh Mittal

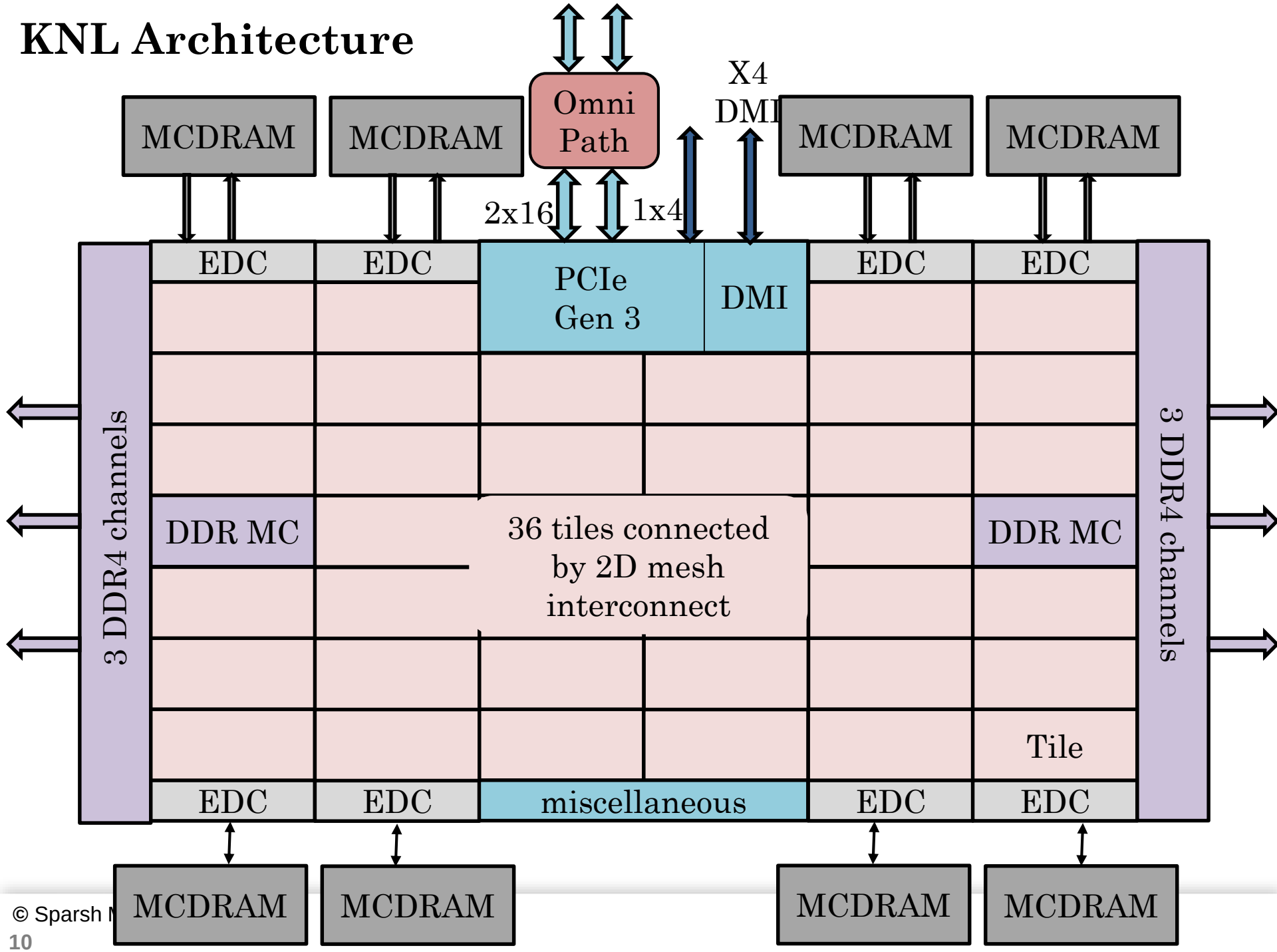
KNC architecture



TD = tag directory
MC = memory controller

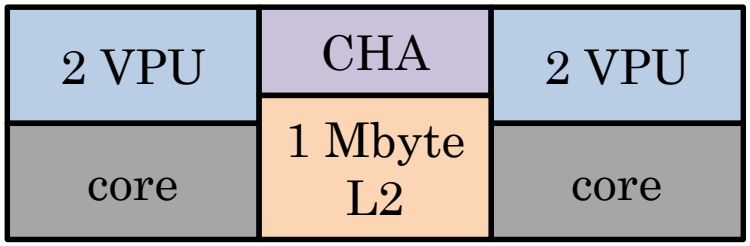
© Sparsh Mittal

KNL Architecture



© Sparsh M

Configuration of each tile



EDC : MCDRAM controllers
CHA : caching/home agent
DDR MC : DDR memory controller
DMI : direct media interface

© Sparsh Mittal

Architectural specifications

	CPU	GPU	Phi			
Model	E5-2697 v2	Tesla K40	3120A	5110P	7250	7295
Microarch.	IvyBridge	Kepler	KNC	KNC	KNL	KNM
Node	22nm	28nm	22nm	22nm	14nm	14nm
Year	2013	2013	2013	2012	2016	2017
# of cores	12	2880	57	60	68	72
Threads	24	30720	228	240	272	288
Clock(GHz)	2.7	0.745	1.1	1.05	1.3	1.5
L1 Cache	32KB	48KB	32KB	32KB	32KB	32KB
L2 cache	256KB	1536KB	512KB	512KB	512KB	512KB
L3 cache	30MB	N/A	N/A	N/A	N/A	N/A
Memory	768GB	12GB	6GB	8GB	384GB	384GB
Bandwidth(GB/s)	59.7	288	240	320	115	115
TDP	130W	235W	300W	225W	215W	320W

© Sparsh Mittal

Feature	KNL	KNM	Broadwell-EP
CPU Model	**7210F**	**7295**	2x E5-2650v4
#{Cores} (HT)	**64** (4x)	**72** (4x)	24 (2x)
Base Frequency	**1.3 GHz**	**1.5 GHz**	2.2 GHz
Max Turbo Freq.	**1.4 GHz**	**1.6 GHz**	2.9 GHz
CPU Mode	Quadrant	Quadrant	*N/A*
TDP	**230 W**	**320 W**	210 W
DRAM Size	96 GiB	96 GiB	256 GiB
↪ Triad BW	**71 GB/s**	**88 GB/s**	122 GB/s
MCDRAM Size	16 GiB	16 GiB	*N/A*
↪ Triad BW	439 GB/s	430 GB/s	*N/A*
MCDRAM Mode	Cache	Cache	*N/A*
LLC Size	**32 MiB**	**36 MiB**	60 MiB
Inst. Set Extension	AVX-512	AVX-512	AVX2
FP32 Peak Perf.	**5,324 Gflop/s**	**13,824 Gflop/s**	1,382 Gflop/s
FP64 Peak Perf.	**2,662 Gflop/s**	**1,728 Gflop/s**	691 Gflop/s

Accounting for #core and core-frequency:
KNM FP32 peak-perf is (13824 * (64/72)* (1.3/1.5)) = 10,649 => 2X of KNL FP32 peak
KNM FP64 peak-perf is (1728 * (64/72)*(1.3/1.5)) = 1331 => 0.5X of KNL FP64 peak

© Sparsh Mittal

Comments

- KNC used in-order core and low frequency => poor single-core performance
- KNL uses out-of-order core and high frequency => high performance

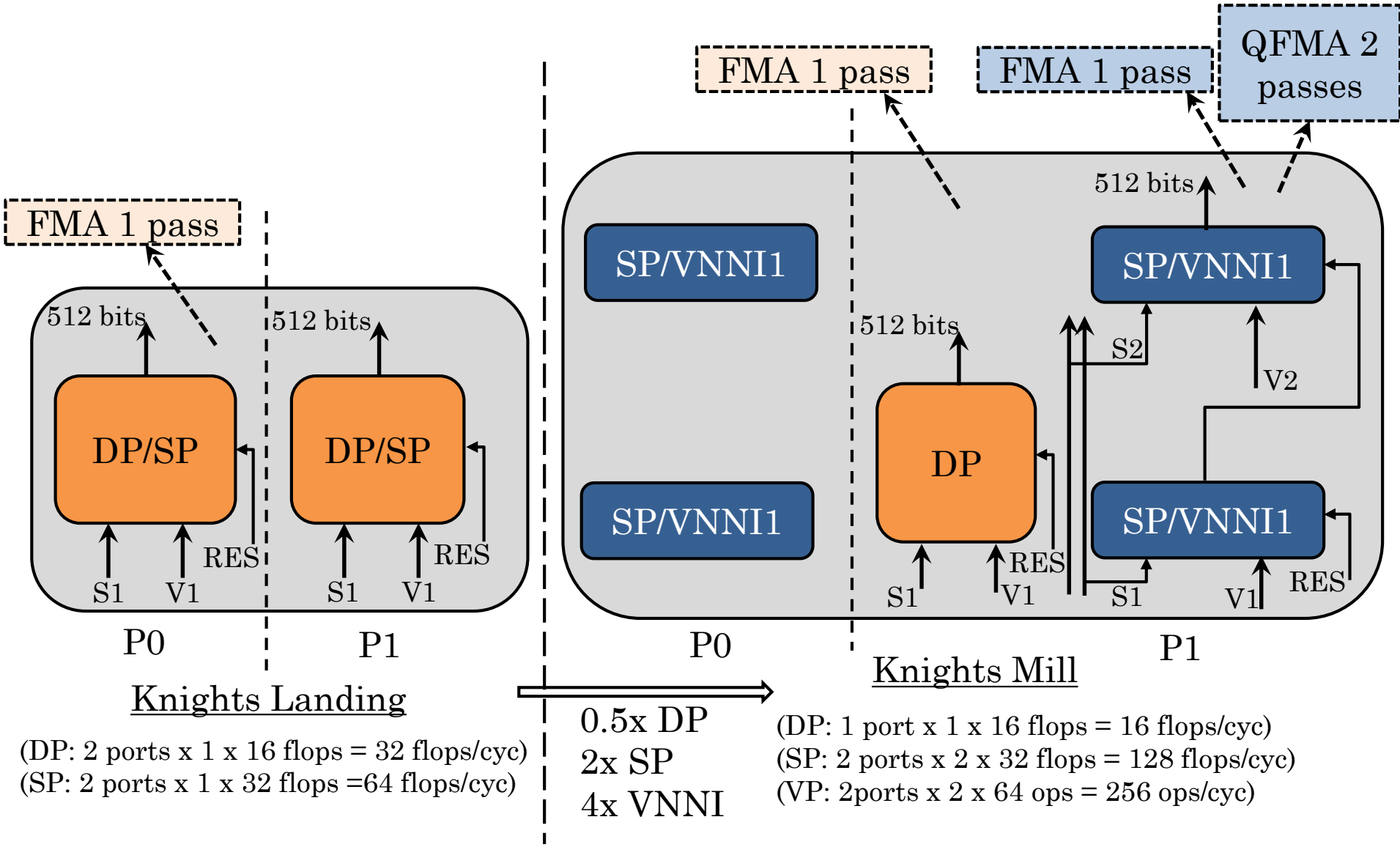
© Sparsh Mittal

KNM Architecture

- KNM chip is identical to KNL in number of cores, frequency, PCIe 3.0 lanes, use of 16GB MCDRAM and 6 DRAM channels.
- KNL: every VPU offers identical 512b interfaces to the AVX-512 unit, supporting SP and DP computations. Half-precision computations are performed by SP block.
- KNM: VPU has asymmetrical ports: instead of SP/DP block, it has separate DP and SP/VNNI blocks and one DP block has been removed.
- Compared to KNL, KNM has 0.5X DP, 2X SP and 4X half-precision performance.

© Sparsh Mittal

Port connections in KNL and KNM



(DP: 2 ports x 1 x 16 flops = 32 flops/cyc)
(SP: 2 ports x 1 x 32 flops =64 flops/cyc)

(DP: 1 port x 1 x 16 flops = 16 flops/cyc)
(SP: 2 ports x 2 x 32 flops = 128 flops/cyc)
(VP: 2ports x 2 x 64 ops = 256 ops/cyc)

https://www.anandtech.com/show/12172/intel-lists-knights-mill-xeon-phi-on-ark-up-to-72-cores-at-320w-with-qfma-and-vnni

© Sparsh Mittal

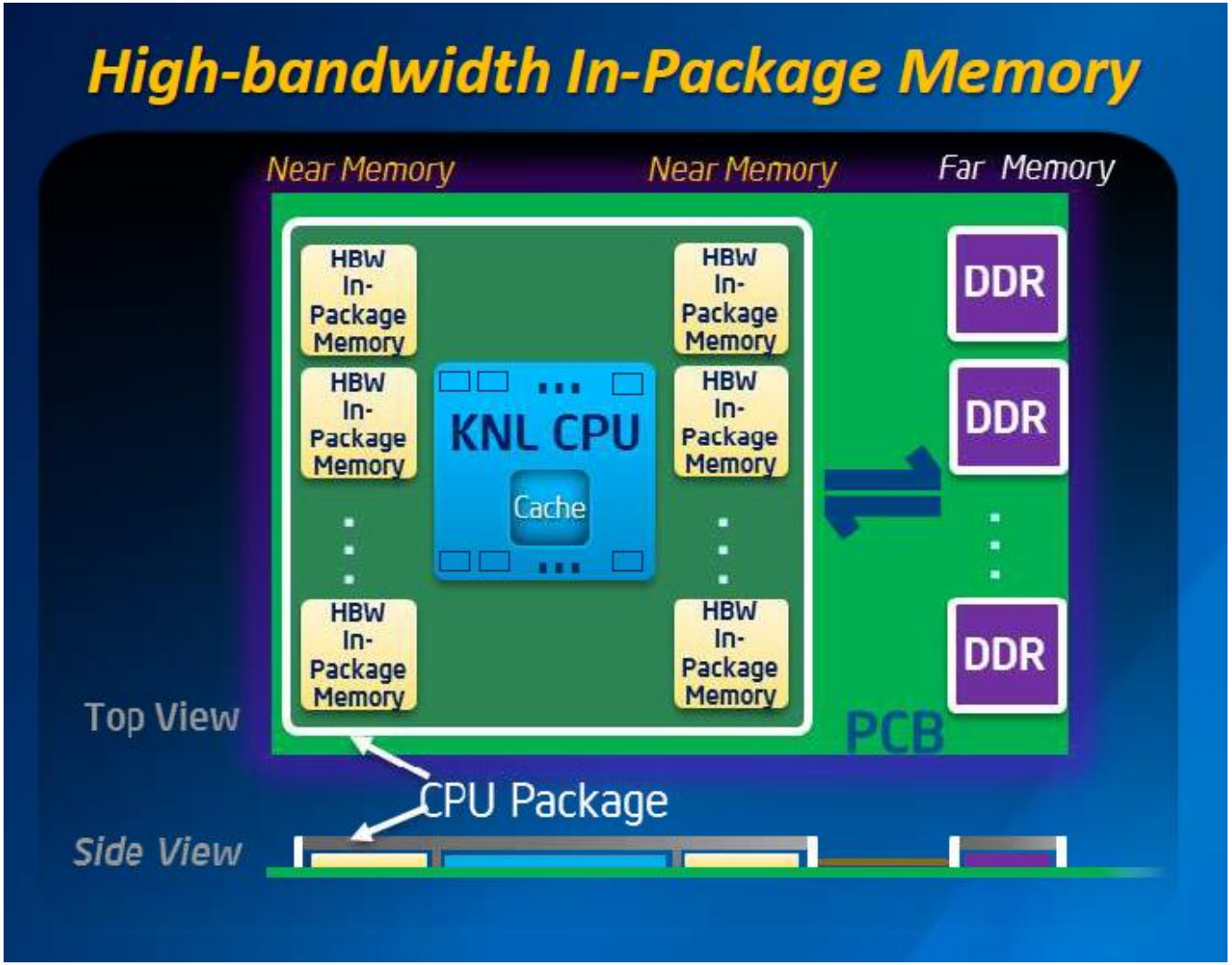
Two types of memories of KNL/KNM

- KNL has two types of memories: MCDRAM and DRAM (also called DDR)

- **MCDRAM:**
  - Micron's 3D "hybrid memory cube"
  - total 16GB (eight 2GB MCDRAM modules)
  - Bandwidth: ~500 GB/s, higher latency than DRAM
- **DRAM:**
  - total 384GB (six channels, each 64GB DDR4)
  - Bandwidth: ~90 GB/s

© Sparsh Mittal

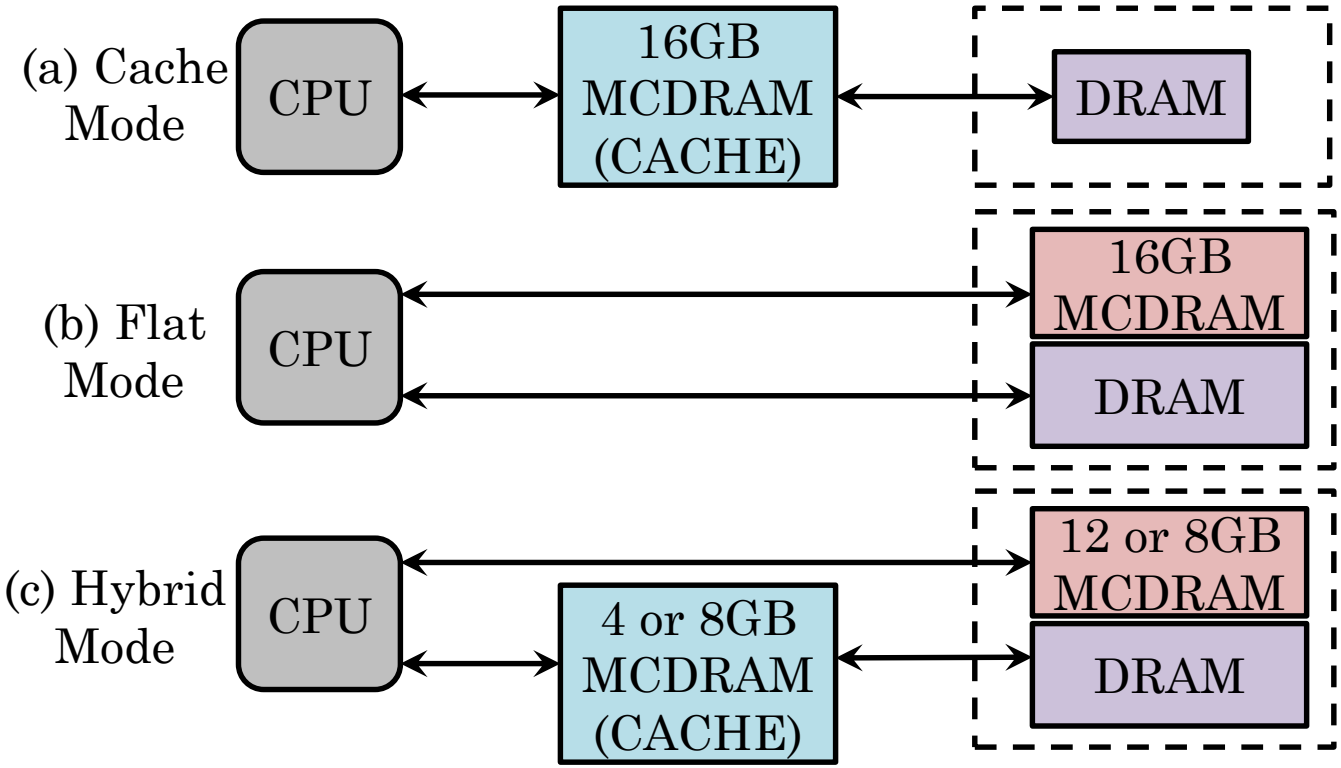
Use of High Bandwidth (HBW) In-Package memory in the Knights Landing



© Sparsh Mittal

Using MCDRAM

- MCDRAM can be used in 3 modes (configured at boot time)



© Sparsh Mittal

Limitation of MCDRAM

- Bandwidth advantage of MCDRAM is realized only when multiple threads access MCDRAM simultaneously
- For bitonic sort, use of MCDRAM provides no benefit over use of DRAM because in bitonic sort, all the cores access memory only in the initial stage, where the size of merged arrays is small.
- In subsequent stages, the number of threads is halved until only one thread is performing the sorting.
- Since bandwidth for a single-thread is same (~8GB/s) in both the memories, MCDRAM provides no advantage over DRAM.

© Sparsh Mittal

Features of Phi

© Sparsh Mittal

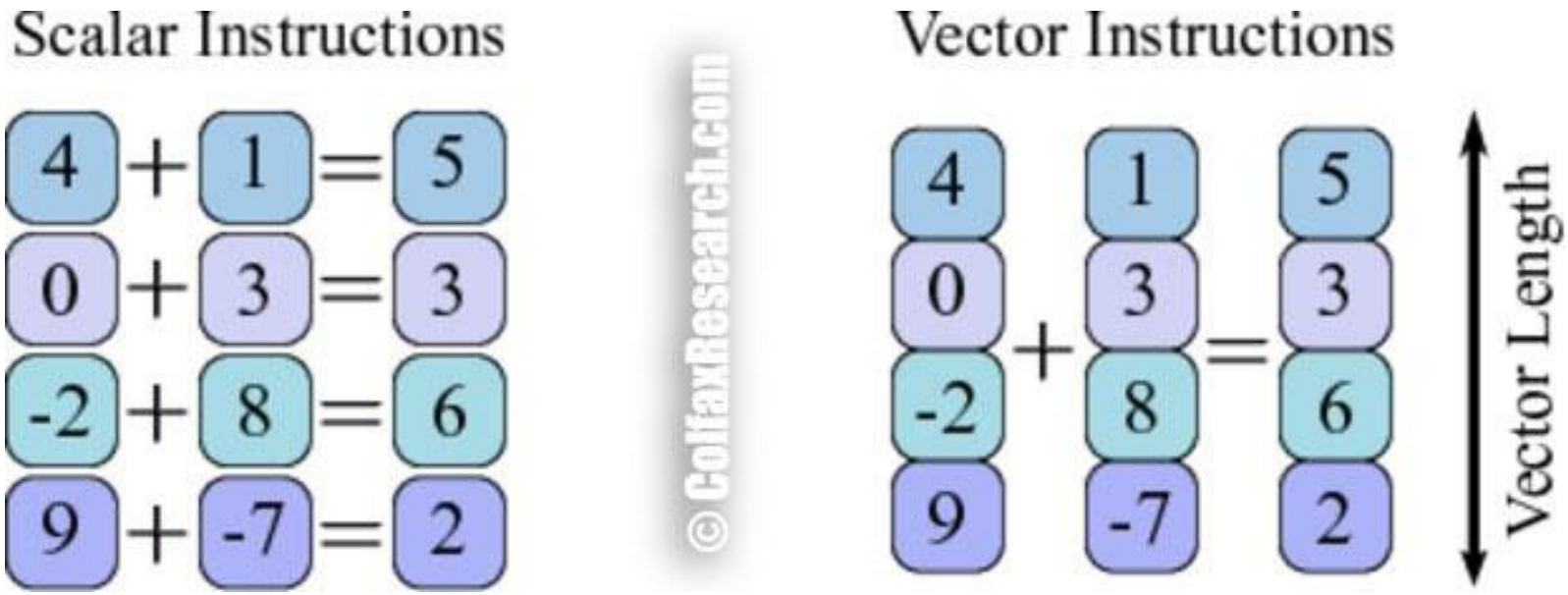
Background: Shared and distributed memory programming

- Shared Memory Programming
  - Threads share memory address space
  - Application ensures no data corruption (Lock/Unlock)

- Distributed memory (Message Passing)
  - Memory is not shared
  - Explicit communication between processes (like sending and receiving emails)

© Sparsh Mittal

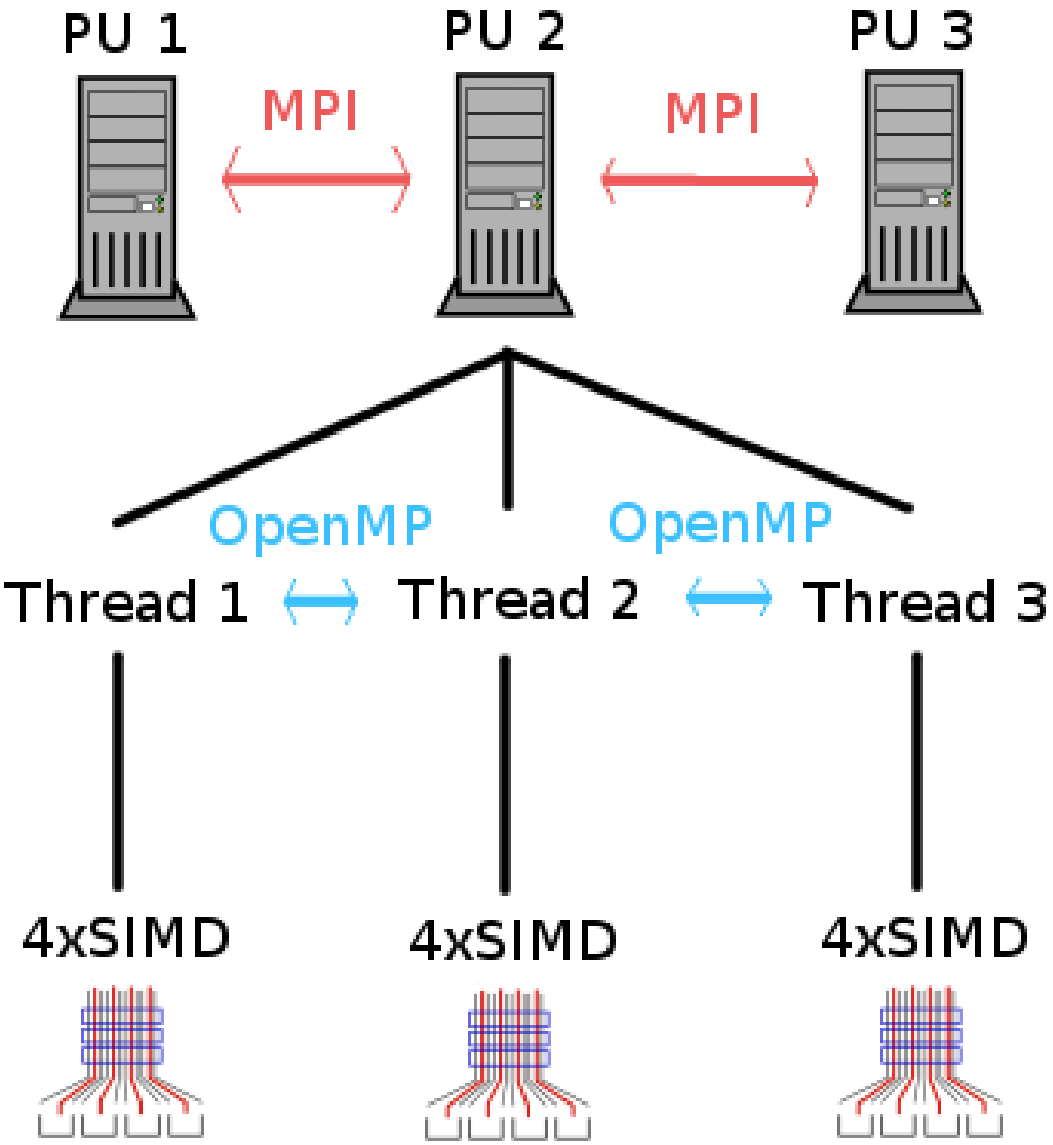
Background: Vectorization



© Sparsh Mittal

https://colfaxresearch.com/knl-avx512/

A parallel program running on three computation units with three threads each with width 4 SIMD unit in each thread



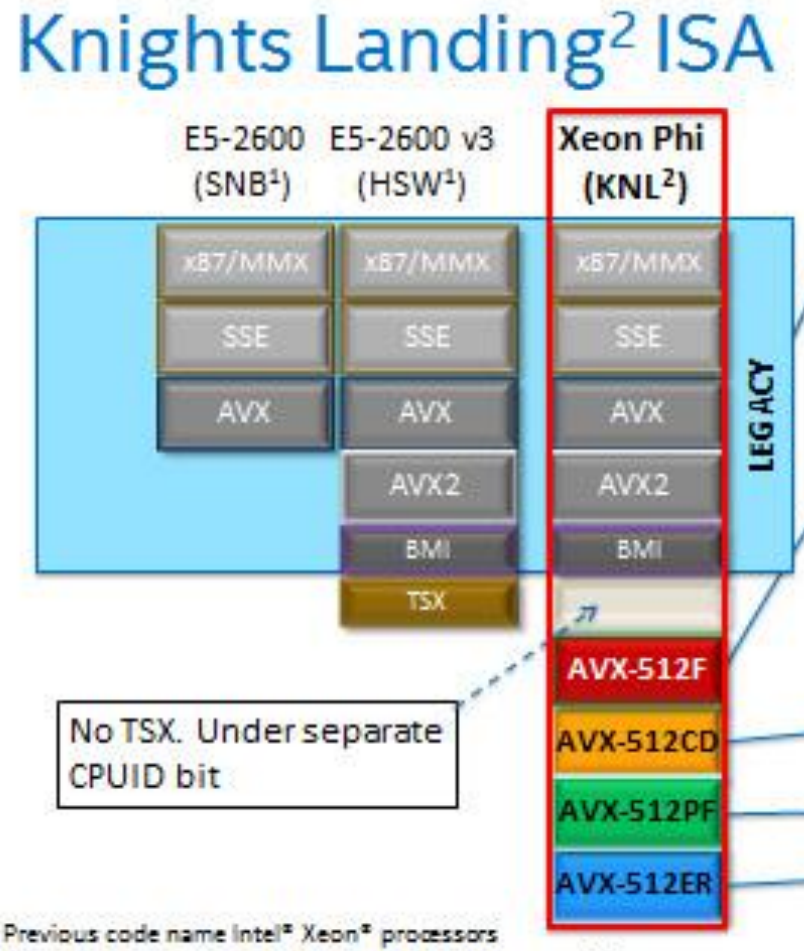
© Spa

Ease of Programming and High-performance

- x86-compatible and can run standard programming languages and APIs such as C/C++/Fortran, MPI, Pthreads and OpenMP.
- Compared to other platforms such as GPU, FPGA and ASIC, Phi incurs much lower overhead of programming and development.
- This is especially beneficial for large-scale legacy codes
- Phi has wide vector units and can allow exploiting thread and data-level parallelism

© Sparsh Mittal

ISA support



KNL implements all legacy instructions

- Legacy binary runs without recompilation
- KNC binary requires recompilation

KNL introduces Intel® Advanced Vector Extensions 512 (Intel® AVX-512)

- 512-bit floating point/integer vectors
- 32 registers and 8 mask registers
- Gather/Scatter

Conflict Detection: Improves vectorization

Prefetch: Gather and scatter prefetch

Exponential and Reciprocal Instructions

[1] Previous code name Intel® Xeon® processors
[2] KNL, Knights Landing = Intel® Xeon Phi™ processor x200

© Sparsh Mittal

Limitations of Phi
and
Challenges in using Phi

© Sparsh Mittal

Need of parallelization

- Need to parallelize over:
  - Node-level (MPI)
  - Thread-level (OpenMP)
  - Data-level (also called vectorization)

  } Easier

- Compared to CPU, Phi has poor serial performance but wide (512 bit) vector units. Hence, on Phi, vectorization is extremely important to achieve high performance
- Generally compiler can auto-vectorize the code, still, user may have to spend huge efforts in some cases

© Sparsh Mittal

Vectorization is difficult for loops with

- multiple exits
- small trip-count
- complex pointer-computations
- indirect memory-access
- data-dependence
- Function calls

© Sparsh Mittal

Tradeoff between multithreading and caching

- To hide high memory latency, Phi needs to run large number of threads which improves core-utilization. But it creates cache contention.
- Parallel processors allocate larger fraction of their resources to compute logic than caches
- Similar to GPU, but unlike CPUs, Phi does not have a large shared last level cache.
- Hence, if the working set of a program does not fit in L2 cache, it has to reside in off-chip memory and not in on-chip L3 cache as in case of CPUs.
  - such a program has poor performance on Phi

© Sparsh Mittal

Control Flow Dependencies

- At any time, all SIMD lanes have to execute the same instructions on different data elements.
- However, in the different branches of an if-else clause, different lanes may execute different instructions, which is not supported by SIMD

© Sparsh Mittal